ORIE 5750: Final Project Report

Alexandra Magnusson
Cornell Tech
am3492@cornell.edu

Victoria Piroian
Cornell Tech
vp324@cornell.edu

Amanda Lu
Cornell Tech
azl8@cornell.edu

1. Abstract

Automated pneumonia detection from chest X-rays can support clinical workflows, but accurately localizing lung opacities remains challenging due to subtle features and class imbalance. We evaluated two deep learning approaches on the RSNA Pneumonia Detection Challenge dataset: a standalone object detection model and a two-stage pipeline combining classification with detection. DenseNet121 slightly outperformed ResNet50 in classification (85.67% accuracy) but showed low recall (52.89%). For localization, Faster R-CNN outperformed YOLOv5n (mAP 0.13215 vs. 0.09722), highlighting the advantage of two-stage detectors for fine-grained medical anomalies. However, cascading the classifier with detection reduced overall performance (mAP $\sim$0.07) due to missed positives, emphasizing the importance of prioritizing high recall in automated screening systems.

2. Introduction

Pneumonia is a common lung infection that is typically diagnosed using chest radiograph (CXR) images. However, detecting pneumonia in these images can be difficult due to subtle visual patterns and typically requires review by highly trained specialists [7]. This makes finding an automated method for pneumonia detection an important and practical application of deep learning in medical imaging.

The RSNA Pneumonia Detection Challenge dataset consists of CXR images collected from multiple sources. Each image is labeled as either pneumonia-positive or negative, and pneumonia-positive images include one or more bounding boxes indicating regions of lung opacity associated with pneumonia. The dataset is highly imbalanced, with a large number of normal images compared to pneumonia cases, which presents additional challenges for both classification and detection tasks.

Many existing methods focus either on classification or on direct object detection. Applying detection models to all of the images can lead to false positives, while classification-only approaches lack localization information. To address this, we use a two-stage pipeline. First, a classification model is used to identify images that are likely to contain pneumonia. These images are then passed to a detection model to locate pneumonia regions using bounding boxes.

For classification, we use ResNet-50 and DenseNet-121. For detection, we evaluate YOLO and Faster R-CNN, allowing us to compare different model architectures within the proposed pipeline. Our goal is to evaluate whether combining classification and detection improves pneumonia detection performance while reducing false positives and maintaining interpretability.

3. Related Work

Several studies have explored pneumonia detection in chest X-ray images, especially with the release of large public datasets such as the RSNA Pneumonia Detection Challenge. Many approaches focus on image-level classification using architectures such as ResNet and DenseNet and commonly report strong performance on pneumonia detection tasks [1]. Most recently, Slimi et al. achieved an accuracy of 99.35% with their custom deep learning model [8], demonstrating the potential for highly accurate automated pneumonia screening in clinical settings.

Other work has investigated object detection models to localize pneumonia regions using bounding box annotations provided in the RSNA dataset [9]. Two-stage detectors like Faster R-CNN have been used due to their localization accuracy, while newer models with YOLO components have been explored to achieve faster inference times. Yao et al. achieved a mean average precision (mAP) score of 39% with their model based on Faster R-CNN [10], showing improvement over previous methods. These studies demonstrate the feasibility of directly predicting pneumonia regions from chest X-ray images and the benefits of combining accuracy and efficiency in detection pipelines.

Beyond pneumonia, similar detection approaches have been applied to other chest X-ray datasets for diseases such as COVID-19. Several studies have used deep learning-based classification and detection models to localize regions of infection or lung abnormalities, demonstrating the general applicability of these techniques to different thoracic

pathologies [3, 4].

Our work follows the general direction of previous studies by evaluating a simple two-stage classification–detection pipeline on the RSNA dataset and comparing commonly used architectures for both tasks.

4. Methods

4.1. Augmentation

To improve model generalization and reduce overfitting, we implemented an offline data augmentation pipeline using the Albumentations library. A diverse set of geometric and photometric transformations was applied to the training images, including horizontal flips with a probability of 0.5, affine transformations consisting of random shifts, scaling, and rotations with a maximum rotation angle of 5 degrees, and random brightness and contrast adjustments applied with a probability of 0.2. These transformations were selected to simulate realistic variations in patient positioning, imaging conditions, and acquisition differences commonly observed in data.

Rather than performing augmentation on-the-fly during training, we adopted an offline augmentation strategy. Augmented samples were generated in advance for approximately one-fifth of the original training set, resulting in a static but expanded dataset. This approach allowed us to maintain consistent training data across experiments and reduced runtime overhead during model training.

For detection-related samples, augmentation was applied jointly to both images and their corresponding bounding box annotations. Albumentations was used to automatically transform bounding box coordinates in accordance with the applied image transformations, ensuring that localization labels remained accurate after augmentation. All augmented images and updated annotations were saved alongside the original preprocessed data, effectively increasing the diversity of the training set while preserving annotation consistency. A representative example of a training batch featuring these augmented images and their corresponding ground-truth bounding boxes is shown in Figure 1.

4.2. Pre Processing

The chest radiograph images in the RSNA Pneumonia Detection Challenge dataset are provided as 1024×1024 DICOM files. To prepare the data for training, we first loaded the DICOM files using the pydicom library and extracted the pixel arrays. The pixel values were normalized to a 0–255 range using min–max scaling and converted into standard image formats.

Since the images are single-channel grayscale X-rays, we converted them into three-channel grayscale images to ensure compatibility with commonly used pre-trained CNN architectures such as ResNet and DenseNet. For the classification models, all images were resized to 224×224 pixels. For the detection models, images were resized to 512×512 pixels to balance computational efficiency and spatial detail.

Ground-truth annotations were extracted from the provided CSV files, including patient IDs and bounding box coordinates (Figure 2). Bounding boxes were proportionally rescaled when images were resized to maintain alignment, and all images and annotations were converted into PyTorch tensors for training and evaluation.

We also performed a 80/20 training-validation split on the training dataset to reserve some data for hyperparameter finetuning, making sure to use a fixed random seed to ensure reproducibility.

4.3. Classification

For the first step of our pipeline, we intend to classify the radiograph images as either containing pneumonia or not. To do this, we decided on testing out two different classification models: ResNet50 and DenseNet121.

4.3.1 ResNet50

First we evaluated ResNet50, a 50-layer convolutional neural network. ResNet addresses the vanishing gradient problem, where performance degrades as depth increases, by introducing residual ("skip") connections that allow gradients to flow more effectively through deep networks [3]. This architecture has proven highly effective in medical imaging tasks, including the detection of lung abnormalities from chest radiographs [3].

Given the relatively limited size of the labeled pneumonia dataset compared to large-scale vision benchmarks, we utilized transfer learning to reduce overfitting and speed up convergence during training. We initialized the model with IMAGENET1K_V2 weights, leveraging feature representations learned from the massive ImageNet dataset. All images were normalized using ImageNet mean and standard deviation to match the pretraining distribution. Data was then loaded using PyTorch DataLoaders with shuffling enabled for the training set and disabled for the validation set. Additionally, to adapt the ImageNet architecture for our binary classification task (Pneumonia vs. Normal), we replaced the final fully connected layer (typically 1000 classes) with a single-neuron linear layer ($N_{out} = 1$).

We trained ResNet50 using the Adam optimizer with Binary Cross Entropy with Logits Loss (BCEWithLogitsLoss), which combines a Sigmoid layer and the BCELoss in a single class for numerical stability. Classification accuracy was computed by setting a threshold on sigmoid outputs at 0.5. Due to fast convergence, training was conducted for 10 epochs, with validation accuracy calculated at the end of each epoch. Checkpoints were saved only when validation accuracy improved, ensuring that the final stored model

represented the best generalization performance across all epochs.

To optimize model performance, we conducted a systematic exploration of learning rates and batch sizes. We tested with learning rates of 0.001, 0.0001, and 0.00003 and batch sizes of 16 and 32, training each configuration for 10 epochs. These learning rates were chosen to cover a range from standard defaults (0.001) to smaller values that allow more stable fine-tuning on a smaller medical imaging dataset. Higher learning rates created a faster initial convergence but occasionally resulted in less stable validation performance, while lower learning rates produced smoother training convergence. Batch sizes were selected based on memory constraints and the trade-off between gradient estimate stability (larger batches) and more frequent weight updates (smaller batches). Smaller batch sizes yielded small improvements in validation accuracy at the cost of increased training time. Validation accuracy served as the primary metric for comparing configurations.

4.3.2 DenseNet121

In addition to ResNet, we implemented DenseNet-121, which differs from ResNet by concatenating feature maps from all previous layers rather than summing them. This densely connected architecture improves feature propagation and reuse, achieving high performance with fewer parameters [12], which is especially useful for small medical imaging datasets [4].

Similar to ResNet, we utilized transfer learning and initialized DenseNet-121 with ImageNet-pretrained weights (IMAGENET1K_V1)to improve generalization and convergence. All images were normalized using ImageNet mean and standard deviation to match the pretraining distribution. We also replaced the original classifier with a binary classification head consisting of a dropout layer (probability 0.3) followed by a single-output linear layer to reduce overfitting.

DenseNet-121 models were trained using the same preprocessing methods, loss function, and evaluation procedure as ResNet50 for comparability. We used the Adam optimizer with BCEWithLogitsLoss, and classification accuracy was computed on the validation set after each epoch.

Based on preliminary experiments, we performed hyperparameter tuning using lower learning rates (0.0001, 0.00003) and batch sizes of 16 and 32. Lower learning rates were critical for stable training, as higher rates produced noisy validation accuracy. Batch sizes were selected to balance gradient stability and memory constraints. A checkpointing system saved the model state with the highest validation accuracy, ensuring the final performance reflected the best generalization.

This consistent training protocol allowed us to attribute performance differences primarily to architectural characteristics and hyperparameter settings rather than differences in methodology.

4.4. Detection

For the second step in our detection pipeline, we intend to take the images classified as containing pneumonia and generate bounding boxes only on those images to increase both accuracy and processing time. For this step, we also tested two different detection models: Faster-RCNN and YOLO.

4.4.1 Faster-RCNN

To address the challenge of accurately localizing pneumonia opacities, we implemented the Faster R-CNN architecture. While single-stage detectors (like YOLO) offer speed advantages, two-stage detectors have historically demonstrated superior accuracy in medical imaging tasks where high precision is critical for diagnosis [10]. Our implementation follows the standard two-stage pipeline consisting of a Region Proposal Network (RPN) and a Fast R-CNN detector.

We used a ResNet-50 backbone pre-trained on ImageNet to extract hierarchical features from the input chest X-rays. To handle pneumonia opacities of varying sizes, we incorporated a Feature Pyramid Network (FPN), allowing the model to effectively detect objects at multiple scales.

The first stage, the Region Proposal Network (RPN), generates candidate bounding boxes likely to contain pneumonia. These proposals are refined and classified in the second stage using features extracted via RoI Align, which preserves spatial information and improves localization accuracy for small regions. The final head predicts both a classification score (Pneumonia vs. Normal) and refined bounding box coordinates.

The model was trained using the SGD optimizer with momentum and weight decay. A StepLR scheduler adjusted the learning rate every few epochs. Training was performed for 10 epochs, and checkpoints were saved based on the best validation performance. During training, a multi-task loss combining classification and bounding box regression was optimized simultaneously:

$$L = L_{cls}^{RPN} + L_{box}^{RPN} + L_{cls}^{ROI} + L_{box}^{ROI} \tag{1}$$

This approach encourages the network to learn features that are simultaneously useful for both recognizing pneumonia and accurately localizing it [11].

4.4.2 YOLOv5n

To complement the high-accuracy two-stage approach of Faster R-CNN, we implemented YOLOv5n, a single-stage detector prioritized for computational efficiency and real-time inference speeds. Unlike region-based detectors that

separate proposal generation and classification, YOLO algorithms directly map image pixels to bounding box coordinates and class probabilities in a unified framework [2]. This architecture allows for significantly reduced latency compared to multi-stage pipelines [6].

We specifically selected the YOLOv5n ("Nano") variant due to its small size, fewer than 2 million parameters, while still maintaining advanced detection capabilities [5]. Its compact architecture and efficient computations made it an attractive option to test whether a fast, single-stage model could complement or rival the two-stage Faster R-CNN in pneumonia localization.

Our implementation utilized the Ultralytics YOLOv5 framework and initialized the model with COCO-pretrained weights, allowing the model to leverage generic visual features. We used the previously preprocessed 512×512 images to balance inference speed and spatial resolution for detecting pneumonia regions. The network was fine-tuned on the RSNA dataset using the provided bounding box annotations, directly predicting both bounding boxes and class probabilities in a single pass.

This setup provided a fast detection baseline that could be compared to our two-stage classification–detection pipeline, allowing us to evaluate the trade-offs between speed and localization accuracy.

5. Results

5.1. Classification

For our classification models, we first evaluated their performance on their own with accuracy as the primary metric. Unlike the detection models which output bounding boxes, the classification model assigns a global binary label (Pneumonia vs. Normal) to each image.

5.1.1 ResNet50

We evaluated the ResNet50 classification model on the held out validation split from our original training dataset. During evaluation, the model outputs a raw logit, which is transformed into a probability score using a Sigmoid activation function. We utilized a standard decision threshold of 0.5, with predictions above this threshold labeled as positive (Pneumonia) and those falling below labeled as negative (Normal).

The results of the hyperparameter tuning experiments are shown in Table 1. The best performing ResNet50 configuration used a learning rate of 0.0001 and a batch size of 32, achieving a validation accuracy of 85.34%. These results suggest that the residual learning framework is highly effective for this medical imaging task.

In terms of specific class metrics, the best performing ResNet50 model achieved a precision of 0.7497 and a recall of 0.5071, culminating in an F1-Score of 0.6050. The model exhibits high precision but moderate recall, indicating a tendency to be conservative predicting pneumonia, likely minimizing false positives at the cost of missing some subtler cases.

Learning Rate	Batch Size	Train Acc.	Val Acc.
0.001	32	0.8487	0.8434
0.0001	32	0.8872	0.8534
0.00003	32	0.9920	0.8456
0.001	16	0.8297	0.8359
0.0001	16	0.8507	0.8498
0.00003	16	0.9897	0.8495

Table 1. ResNet50 Hyperparameter Tuning Results

Additionally, we observed that most configurations tended to overfit rapidly, with training accuracy exceeding 97% by epoch 10 for smaller learning rates. In contrast, models trained with a learning rate of 0.001 achieved training accuracies of around 85%. This suggests that the larger learning rate resulted in coarser parameter updates, limiting the model's ability to fully fit the training data within the fixed number of epochs, whereas smaller learning rates allowed for more precise optimization.

5.1.2 DenseNet121

We also evaluated the performance of the DenseNet121 classification model on the validation dataset. Similar to ResNet50, the model outputs a raw logit that is converted to a probability score using a Sigmoid activation function. A decision threshold of 0.5 was applied to obtain binary predictions for pneumonia versus normal cases.

The results of the hyperparameter tuning experiments are shown in Table 2. The best performing DenseNet121 configuration used a learning rate of 0.00003 and a batch size of 32, achieving a validation accuracy of 85.67%. Overall, DenseNet121 demonstrated slightly higher validation accuracy than ResNet50 under comparable training conditions.

Learning Rate	Batch Size	Train Acc.	Val Acc.
0.0001	32	0.8894	0.8548
0.00003	32	0.8919	0.8567
0.0001	16	0.8572	0.8565
0.00003	16	0.9859	0.8534

Table 2. DenseNet-121 Hyperparameter Tuning Results

Across configurations, lower learning rates resulted in more stable validation performance, while batch size had a smaller impact on overall accuracy. Notably, the smallest learning rate combined with a smaller batch size led to very high training accuracy without corresponding validation gains, indicating overfitting.

To further analyze the model's clinical utility, we examined the trade-off between Precision and Recall. The model achieved a Precision of 0.7500 and a Recall of 0.5289, resulting in an F1-Score of 0.6203. These metrics suggest that while the model is highly precise (when it predicts pneumonia, it is likely correct), it is more conservative in its sensitivity, potentially missing some subtle cases of opacity.

5.2. Detection

We first evaluated each detection model independently to establish baseline localization performance before integrating them into the two-stage classification–detection pipeline. Detection performance was measured using mean Average Precision (mAP). We report mAP@50 to capture performance under a more lenient localization threshold, mAP@50–95 to provide a broader view of detection quality across increasingly strict criteria, and mAP@40–75 to remain consistent with the official evaluation metric used in the Kaggle competition.

5.2.1 Faster R-CNN Results

We evaluated the performance of our Faster R-CNN model on the RSNA Stage 2 test dataset, which consists of 3,000 unlabeled images. Inference was performed using the model checkpoint that achieved the highest validation accuracy during the training phase.

To maintain consistency with our training pipeline, we used the previously pre-processed test images that were resized to 512×512 pixels before being fed into the network. The model outputs a set of proposed bounding boxes, each with an associated "objectness" confidence score indicating the likelihood of pneumonia.

To reduce false positives and ensure that only high-confidence detections were submitted, we applied a post-processing confidence threshold of 0.3. Any predicted bounding box with a confidence score below this threshold was discarded. The remaining bounding box coordinates were then rescaled from the 512×512 input space back to the original dimensions of the source DICOM images. This rescaling step is critical, as evaluation is performed using the original image resolution in the Kaggle competition.

The training progression, illustrating the model's convergence over 10 epochs as indicated by the training and validation loss curves, is presented in Figure 7. The final predictions were formatted into a CSV submission file and evaluated against the ground truth via the Kaggle competition leaderboard. Our Faster R-CNN model achieved a mAP@40–75 score of 0.13215. For reference, the highest score achieved in the competition was 0.25475, highlighting the challenging nature of this detection task. This score reflects the model's ability to localize pneumonia regions while balancing detection confidence and spatial accuracy under the competition's evaluation protocol. The Precision-Recall curve evaluated at an IoU threshold of 0.5, which provides a detailed visualization of the model's detection performance and corresponds to an Average Precision (AP) of 0.42, is shown in Figure 8.

5.2.2 YOLOv5n Results

We evaluated the performance of the single-stage YOLOv5n model on the same pre-processed RSNA Stage 2 test set, with images resized to 512×512 pixels. Similar to Faster R-CNN, the model outputs a set of predicted bounding boxes with associated confidence scores, which were then rescaled back to the original image dimensions.

Redundant detections were removed using the YOLO's built-in Non-Maximum Suppression (NMS) with an Intersection over Union (IoU) threshold of 0.45. We experimented with different post-processing confidence thresholds because using the default value of 0.25 resulted in many missed ground-truth boxes on the validation set. The resulting mAP scores for different thresholds are shown in Table 3.

Confidence	mAP@50	mAP@50-95
0.25	0.375	0.191
0.20	0.375	0.186
0.15	0.37	0.19
0.10	0.368	0.173

Table 3. YOLOv5n mAP score on validation set with different confidence thresholds

Based on these results, we selected to use a confidence threshold of 0.15, which maintained comparable validation performance to the 0.25 threshold while producing more bounding boxes. Upon submission of our test predictions to the Kaggle evaluation server, the YOLOv5n model achieved a Mean mAP@40–75 of 0.09722. This score provides a baseline for single-stage detection performance, showing that YOLOv5n can detect pneumonia regions, albeit at a lower mAP than the Faster R-CNN model. The training progression, illustrating the convergence of loss functions and the improvement of mean Average Precision (mAP) over 50 epochs, is visualized in Figure 6. The Precision-Recall curve, which details the trade-off between precision and recall at varying confidence thresholds and corresponds to the reported mAP, is presented in Figure 5.

A qualitative comparison of the model's detection performance is provided in Figure 3 and Figure 4, which display ground-truth annotations alongside the model's predicted bounding boxes and confidence scores for a representative validation batch. This performance gap highlights the trade-off between the architectural efficiency of the "Nano" model (optimized for speed and low parameter count) and the fine-grained localization capabilities of the more complex, two-stage Faster R-CNN architecture, particularly when detecting subtle medical anomalies.

5.3. Full Pipeline

Combining our best-performing classification model, DenseNet-121 (with a learning rate of 0.00003 and batch size of 32), with YOLOv5n into a full two-stage detection pipeline, we evaluated its performance on the RSNA Stage 2 test set. After submitting our results to Kaggle, the pipeline achieved a mean average precision (mAP) score of 0.07563. This score did not improve over the standalone YOLOv5n detection model, which achieved an mAP of 0.10375.

We also tested the same DenseNet-121 model in a pipeline with our FasterRCNN model for comparison. After submitting our resulting CSV to Kaggle, we achieved an mAP score of 0.06887. This score was also worse than the standalone FasterRCNN model, which achieved an mAP of 0.13215.

The lower performance of the combined pipeline is likely due to the limited accuracy of the classification model, which may have filtered out images containing pneumonia and reduced the effective dataset available for detection.

Model	mAP@40-75
Faster R-CNN (baseline)	0.13215
YOLOv12n (baseline)	0.10375
Faster R-CNN (pipeline)	0.06887
YOLOv12n (pipeline)	0.07563
Kaggle Competiton Best Score	0.25475

Table 4. Final Kaggle Competition Results for each model

6. Discussion

The different models and experiments implemented and discussed in this report showcased the trade-offs between model complexity, computational efficiency, and clinical sensitivity in pneumonia detection. Our main finding was that the two-stage detection architecture, Faster R-CNN, substantially outperformed both the single-stage YOLOv5n model, and the proposed classification–detection pipeline. This suggests that, for tasks requiring precise localization of subtle abnormalities, like pneumonia detection through bounding boxes, the sophistication of the architecture used is much more important than the raw speed of inference.

In the classification experiments, DenseNet121 slightly outperformed ResNet50, achieving a validation accuracy of 85.67%, compared to ResNet50's 85.34%. This result supports the hypothesis that DenseNet's dense connectivity pattern, which promotes feature reuse across layers, is well suited for medical imaging tasks where labeled training data is limited [12, 4]. Despite these high accuracy results, both models suffered from a significant imbalance between precision (0.75) and recall (0.53). This indicates that the classifiers adopted a conservative prediction strategy, limiting false positives, regular images labeled pneumonia, but failing to identify nearly half of true pneumonia cases. This behavior is consistent with the class imbalance in our medical datasets, and combined with the difficulty of distinguishing between subtle opacity from normal lung tissue, creates a challenge that has also been observed in prior studies using ResNet-based architectures [3].

For the object detection task, Faster R-CNN achieved the highest mean average precision (mAP) of 0.13215, outperforming YOLOv5n, which achieved an mAP of 0.09722. This performance gap is supported by existing literature that shows that while single-stage detectors like YOLO excel in inference speed and are well suited for real-time applications [6, 2], they often underperform with respect to two-stage detectors in terms of localization accuracy [10]. The Region Proposal Network (RPN) used in Faster R-CNN enabled a more focused and refined search over bounding box candidate regions, which is particularly important for detecting pneumonia opacities as they vary widely in size, shape, and location. In contrast, the "Nano" variant of YOLOv5, although it can be highly efficient when used with less than two million parameters [5], it lacks the modeling capacity needed to capture the specific details and texture related hints that need to be identified in order to distinguish true opacities from naturally occurring structures in the images such as ribs.

Contrary to our initial beliefs, the full two-stage pipeline, which combined the best-performing classifier (DenseNet121) with each detector model, resulted in a significantly worse performance, with the mAP values dropping to 0.07563 for YOLOv5n and 0.06887 for Faster R-CNN. We originally believed that filtering out normal images prior to detection would reduce false positives and simplify the detection task. However, the pipeline instead had even more errors that occurred. The classifier's recall of 0.5289 effectively became a bottleneck, and eliminated approximately 47% of true pneumonia cases before they could be analyzed by the detector. This led us to conclude that when designing machine learning systems for medical systems it is important to optimize a filtering step, if used, for a near-perfect recall rather than overall accuracy, as missed cases at early stages cannot be recovered downstream.

Future work and investigation on this topic should therefore prioritize improving sensitivity at the classification stage. Some promising ideas include the using weighted loss functions that penalize false negatives, or an ensemble approaches that combine multiple architectures to improve the model's resistance to false negatives. Additionally, incorporating attention mechanisms, similar to what was attempted in recent studies [8, 1], may allow models to focus more on relevant regions of the images, giving it a potential advantage over standard global pooling methods.

7. Appendix

7.1. Link to Dataset

We used the Kaggle data set of pneumonia images below for this report. https://www.kaggle.com/competitions/rsna-pneumonia-detection-challenge/overview

7.2. Disclaimer

All project work was shared evenly across the three team members, with frequent in-person meetings to enable cross-collaboration.

7.3. Code Descriptions

7.3.1 Preprocessing

The notebook Preprocessing.ipynb implements the data ingestion and preprocessing pipeline for the RSNA Pneumonia Detection Challenge (Stage 2) dataset. The script utilizes pydicom for handling medical imaging files and albumentations for image transformations.

The pipeline begins by extracting raw pixel arrays from DICOM files, normalizing them to the range $[0, 255]$, and converting them into PIL images. These images are resized into an intermediate resolution of 512×512 and a final input size of 224×224, converted to 3-channel grayscale to satisfy the input requirements of standard CNN architectures. Concurrently, ground truth bounding box coordinates $(x, y, \text{width}, \text{height})$ from stage_2_train_labels.csv are rescaled to align with the resized image dimensions.

To enhance model robustness, the pipeline generates synthetic training examples by applying random horizontal flips, affine transformations (shift, scale, rotate), and brightness/contrast adjustments to a subset of the training data. Binary classification labels are derived for each patient ID, assigning a target of 1 if pneumonia bounding boxes are present and 0 otherwise. Finally, the processed images are serialized as PNG files, and the corresponding metadata are exported to resized_bboxes_512.csv and classification_labels_224.csv for efficient retrieval.

7.3.2 Classification

The notebook ClassificationModels.ipynb implements the training and evaluation pipeline for binary classification using deep learning. The script initializes a PyTorch environment to load pre-processed training and validation images, matching them with labels from CSV files. It applies standard computer vision transformations, including resizing inputs to 224×224 pixels and normalizing pixel intensity using ImageNet mean and standard deviation statistics.

The architecture definition section constructs two distinct Convolutional Neural Networks (CNNs): ResNet50 and DenseNet121. Both models utilize transfer learning by loading pre-trained ImageNet weights. The final classification layers of both networks are modified to output a single logit for binary prediction, with the DenseNet implementation including an optional dropout layer for regularization. The training loop utilizes the BCEWithLogitsLoss function and the Adam optimizer.

To optimize performance, the notebook executes a systematic hyperparameter search over 10 epochs for various configurations. It iterates through combinations of learning rates (specifically 1×10^{-3}, 1×10^{-4}, and 3×10^{-5}) and batch sizes (16 and 32). A checkpointing mechanism evaluates the model after each epoch, saving the model weights to disk only when the validation accuracy surpasses the previous best metric.

7.3.3 FasterRCNN

The notebook Faster-RCNN.ipynb establishes an object detection pipeline using the PyTorch framework. It defines a custom dataset class, PneumoniaDataset, which ingests preprocessed images and bounding box annotations from CSV files. This class specifically filters the dataset to focus on positive pneumonia cases during training and formats the target dictionaries, containing boxes, labels, and area, to strictly adhere to the input requirements of the torchvision detection library.

The architecture utilizes a Faster R-CNN model backed by a ResNet-50-FPN feature extractor pre-trained on the COCO dataset. The standard predictor head is replaced with a FastRCNNPredictor configured for two classes (background and pneumonia). Training is executed over 10 epochs using an SGD optimizer with momentum and weight decay, accompanied by a learning rate scheduler to decay the learning rate periodically.

For evaluation and inference, the script calculates the Intersection over Union (IoU) metric to assess bounding box accuracy against ground truth data. It implements post-processing steps including Non-Maximum Suppression (NMS) with a defined threshold to eliminate overlapping proposals and score filtering to remove low-confidence predictions. Finally, the model generates prediction strings in the specific format required for the competition submission file.

7.3.4 YOLO

The notebook YOLO.ipynb implements a single-stage object detection pipeline utilizing the YOLOv8 (You Only Look Once) architecture via the Ultralytics library. The script begins by configuring the training environment through a YAML file that defines the dataset directory structure and class mappings. The model is initialized using the YOLOv8-Nano (yolov8n.pt) variant with pre-trained

weights to leverage transfer learning for the pneumonia detection task.

The training process is executed over 20 epochs with an input image resolution of 224 pixels and a batch size of 32. The optimizer is explicitly set to Adam with an initial learning rate of 0.001, while other hyperparameters such as dropout are configured to defaults or specific zero values to suit the dataset size. The pipeline automatically saves training metrics and model checkpoints to a designated run directory for later retrieval.

For the inference phase, the notebook loads the best-performing model weights to generate predictions on the unseen test set. The detection parameters include a confidence threshold of 0.25 and an Intersection over Union (IoU) threshold of 0.45 to refine the bounding box proposals. Finally, the script iterates through the detection results to format the confidence scores and coordinates into prediction strings, generating a submission CSV file that accounts for both positive detections and negative cases.

7.3.5 Training and Validation Splitting

The notebook train_val_splitting.ipynb executes the data partitioning strategy to establish distinct training and validation sets for model development. It begins by loading the master classification labels and performing a stratified random split using Scikit-Learn, ensuring that 20% of the data is held out for validation while preserving the original ratio of positive to negative pneumonia cases. The resulting lists of patient IDs are saved as separate CSV files to define the ground truth for each subset.

Following the logical split, the script manages the physical organization of the image data. It defines a function to programmatically retrieve the specific image files assigned to the training and validation groups and compresses them into dedicated ZIP archives (train_224.zip and val_224.zip). This step ensures that the file structure remains portable and efficient for subsequent loading during the training phases.

Finally, the notebook aligns the object detection metadata with the new dataset partition. It filters the master CSV containing resized bounding box coordinates against the generated patient ID lists. This process exports separate bounding box CSV files for the training and validation splits, ensuring that the object detection models are trained and evaluated on the exact same patient cohorts as the classification models.

7.3.6 Evaluation

The notebook evaluate.ipynb performs the quantitative assessment of the model's predictive performance on the test dataset. It begins by ingesting two primary data sources: the model's generated submission file containing prediction confidence scores and the ground truth CSV file containing the actual binary targets for the test subjects. The script merges these datasets based on unique patient IDs to ensure accurate alignment between predictions and true outcomes.

To measure classification efficacy, the notebook utilizes the Scikit-Learn library to compute a comprehensive suite of metrics, including Accuracy, Precision, Recall, and the F1-Score. It specifically calculates the Area Under the Receiver Operating Characteristic Curve (ROC-AUC) to evaluate the model's ability to distinguish between classes across different threshold settings. Additionally, the pipeline generates a confusion matrix, providing a visual breakdown of true positives, false positives, true negatives, and false negatives to diagnose specific error modes.

7.4. Figures

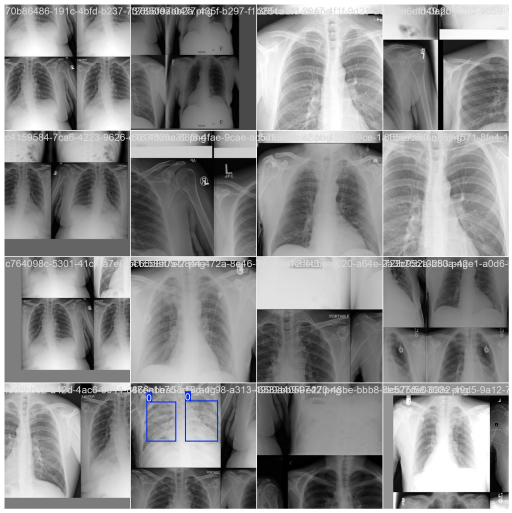

Figure 1. Visualization of the initial training batch (train_batch0). The grid displays a sample of chest X-ray images with applied mosaic data augmentation and resizing. Blue bounding boxes indicate the ground truth annotations for class 0 (pneumonia).

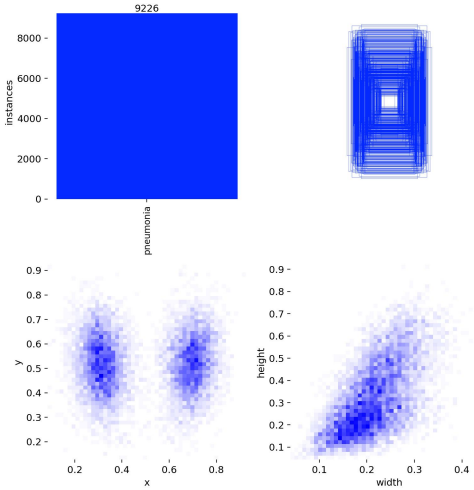

Figure 2. Analysis of the training dataset labels. The subplots illustrate the instance count for the pneumonia class (top-left), bounding box visualization (top-right), spatial distribution of bounding box centers showing bilateral lung positioning (bottom-left), and the correlation between box width and height (bottom-right).

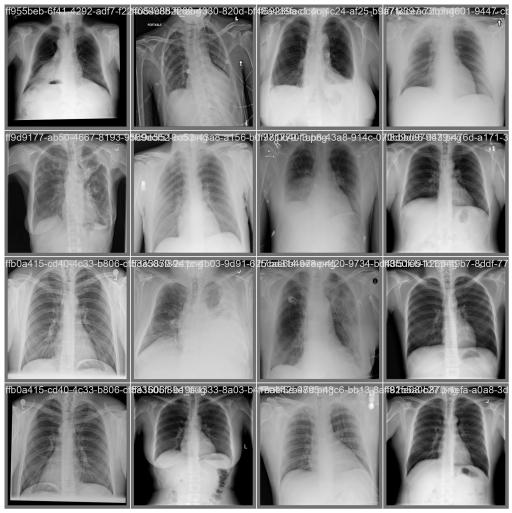

Figure 3. Visualization of the first validation batch (val_batch0_labels). This grid displays a sample of chest X-ray images from the validation set used to evaluate model performance. Unlike the training batches, these images are presented without mosaic augmentation to reflect standard inference conditions.

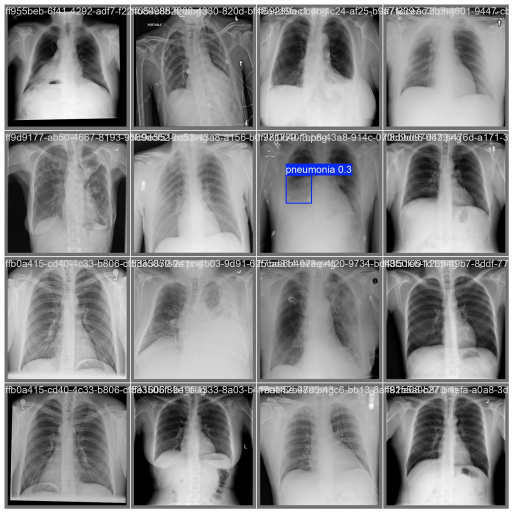

Figure 4. Visualization of model predictions on the first validation batch (val_batch0_pred). This figure displays the bounding boxes predicted by the model along with their confidence scores (e.g., ”pneumonia 0.3”). Comparing these predictions with the ground truth labels in Figure 3 allows for a qualitative assessment of the model’s detection accuracy.

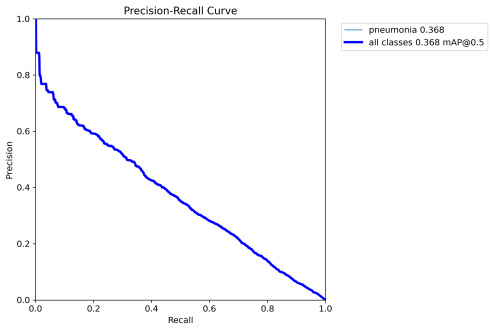

Figure 5. Precision-Recall Curve. This plot visualizes the trade-off between precision and recall for the pneumonia detection model. The area under the curve corresponds to a mean Average Precision (mAP) of 0.368 at an IoU threshold of 0.5 (mAP@0.5), providing a summary metric of the model's overall detection performance.

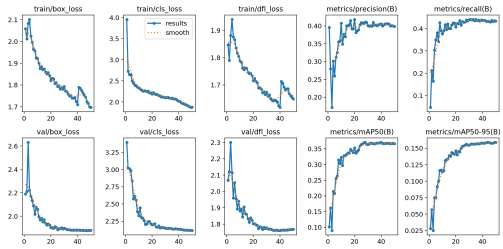

Figure 6. Training and validation metrics over 50 epochs. The curves demonstrate the convergence of the model, showing a steady decrease in training and validation losses (Box, Objectness/Class, and DFL) alongside improvements in key performance metrics such as Precision, Recall, and Mean Average Precision (mAP) at varying IoU thresholds.

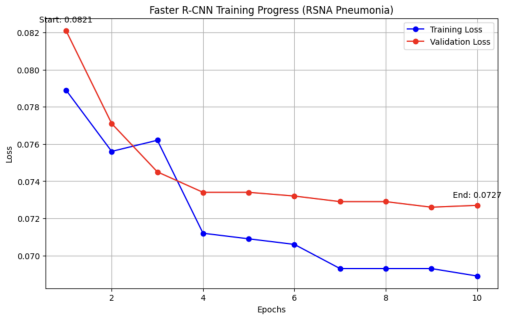

Figure 7. Training and validation loss curves for the Faster R-CNN model over 10 epochs, illustrating the model's convergence on the RSNA Pneumonia dataset.

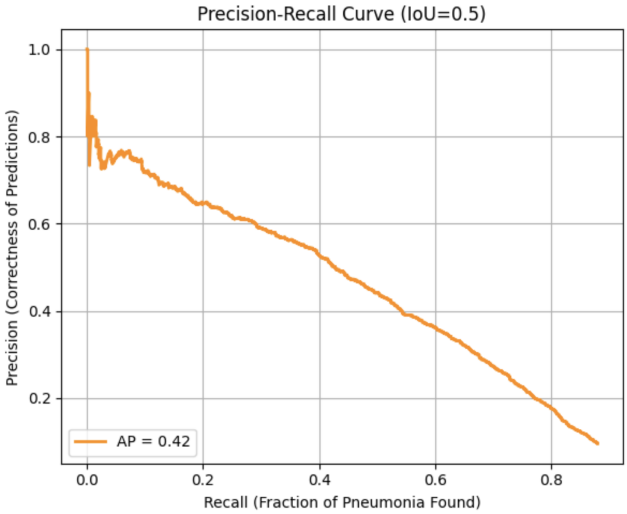

Figure 8. Precision-Recall curve evaluated at an Intersection over Union (IoU) threshold of 0.5, showing an Average Precision (AP) of 0.42.

7.5. Kaggle Results

submission.csv Complete (after deadline) · 2d ago	0.13215	0.04952

Figure 9. FasterRCNN Kaggle Score

Submission and Description	Private Score	Public Score	Selected
yolo_submission.csv Complete (after deadline) · 2d ago	0.10375	0.09722	☐

Figure 10. YOLO Kaggle Score

Submission and Description	Private Score	Public Score	Selected
rcnn_combined_submission.csv Complete (after deadline) · now	0.06887	0.07301	☐

Figure 11. Pipeline RCNN Kaggle Score

yolo_combined_submission.csv Complete (after deadline) · 7h ago	0.07563	0.06547	☐

Figure 12. Pipeline YOLO Combined Kaggle Score

References

[1] Qian An, Wei Chen, and Wen Shao. "A Deep Convolutional Neural Network for Pneumonia Detection in X-ray Images with Attention Ensemble". In: *Diagnostics* 14.4 (2024), p. 390. DOI: 10.3390/diagnostics14040390. URL: https://doi.org/10.3390/diagnostics14040390.

[2] Tausif Diwan, G Anirudh, and Jitendra V Tembhurne. "Object detection using YOLO: challenges, architectural successors, datasets and applications". In: *multimedia Tools and Applications* 82.6 (2023), pp. 9243–9275.

[3] Marwa Elpeltagy and Hany Sallam. "Automatic prediction of COVID- 19 from chest images using modified ResNet50". In: *Multimedia tools and applications* 80.17 (2021), pp. 26451–26463.

[4] Najmul Hasan et al. "DenseNet convolutional neural networks application for predicting COVID-19 using CT image". In: *SN computer science* 2.5 (2021), p. 389.

[5] Muhammad Hussain. "Yolov1 to v8: Unveiling each variant–a comprehensive review of yolo". In: *IEEE access* 12 (2024), pp. 42816–42833.

[6] Peiyuan Jiang et al. "A Review of Yolo algorithm developments". In: *Procedia computer science* 199 (2022), pp. 1066–1073.

[7] Radiological Society of North America. *RSNA Pneumonia Detection Challenge*. https://www.kaggle.com/competitions/rsna-pneumonia-detection-challenge. Accessed: 2025-12-01. 2018.

[8] H. Slimi, A. Balti, S. Abid, et al. "Trustworthy pneumonia detection in chest X-ray imaging through attention-guided deep learning". In: *Scientific Reports* 15 (2025), p. 40029. DOI: 10.1038/s41598-025-23664-x. URL: https://doi.org/10.1038/s41598-025-23664-x.

[9] L. Wu, J. Zhang, Y. Wang, et al. "Pneumonia detection based on RSNA dataset and anchor-free deep learning detector". In: *Scientific Reports* 14 (2024), p. 1929. DOI: 10.1038/s41598-024-52156-7. URL: https://doi.org/10.1038/s41598-024-52156-7.

[10] Shuo Yao et al. "Pneumonia Detection Using an Improved Algorithm Based on Faster R-CNN". In: *Computational and Mathematical Methods in Medicine* 2021 (2021), p. 8854892. DOI: 10.1155/2021/8854892. URL: https://doi.org/10.1155/2021/8854892.

[11] Changzheng Zhang, Xiang Xu, and Dandan Tu. "Face detection using improved faster rcnn". In: *arXiv preprint arXiv:1802.02142* (2018).

[12] Ke Zhang et al. "Multiple feature reweight densenet for image classification". In: *IEEE access* 7 (2019), pp. 9872–9880.